

CS 747: Programming Assignment 2

Margav Mukeshbhai Savsani

200050072

1. Task 1 - MDP Planning Algorithms

I have used two helper functions commonly for all the three subtasks one of which for a given policy and MDP returns the value function for that policy and the other function takes as input a value function and MDP and computes the action value function (considering that the given value function is value function for a policy) and for all states returns the maximum value of action value function for that state and the corresponding best action. I have commented the code well to explain how the functions perform the computation.

(a) Value Iteration

I implemented the algorithm as was mentioned in the class. I initialised the initial guess of value function by random numbers and used a factor of 0.0000000001 to decide whether I have converged to the correct value function.

(b) Howard's Policy Iteration

I implemented the algorithm as was mentioned in the class. I initialised the initial policy randomly. If for a state there are multiple improvable actions I took the one with maximum $Q(s,a)$ and if there are multiple such actions then the one which is selected by numpy's argmax function is taken. Also to check if some action is an improvable action for a state I considered a difference of 0.0000000001 between the state's value and $Q(s,a)$ to decide. I used to believe Howard's Policy Iteration is better as compared to Value Iteration but this made me realise since we are implementing them on a computer both come out to be the same. Although still I have selected Howard's Policy Iteration as the default algorithm .

(c) Linear Programming

I implemented the algorithm as was mentioned in the class with n variables and nk constraints. I fed the constraints and minimization condition to the solver and got the value function and using it found the corresponding optimal policy. It turns out to be the slowest among the 3 algorithms as it takes a lot of time to encode the problem in pulp.

2. Task 2 - MDP for Cricket Game

(a) Encoder

Given n states in the statefile I created a MDP with $2n+2$ states where for each brrr I added a suffix A or B to indicate who is currently on strike and the other two states are the terminal states "WON" and "LOST" marking the end of episode(the first n states for A, after that n states for B and last 2 states for "WON" and "LOST"). All the transitions give a reward of 0 except for the ones that go to state "WON" which yield a reward of 1. A has 5 actions {0, 1, 2, 4, 6} and for B I allowed a single action {8}. I iterated over the first $2n$ (non terminal) states and found out the corresponding runs, balls and person on strike and found out all those states which are reachable from the current state and added the corresponding information to the Transition Probability and Reward matrices which I maintained. I maintained dictionaries to convert string states to whole number states of the MDP and vice versa. I also maintained dictionaries to convert the actions mentions above to corresponding actions in MDP and vice versa. Finally I printed the values from the transition and reward matrices appropriately in the required format.

(b) Decoder

In the decoder I created the exactly same dictionaries as in encoder and hence was able to convert whole number states and actions given by planner.py to corresponding states and actions given in the question. If from a state there is zero probability for winning the planner might output any action and hence if value function for a state is zero I changed the output action to 0 as that was the notation used in given policyfile

3. Analysis

I have also submitted the analysis file to regenerate the plots if needed.

Usage:python analysis.py -policyfile policyfilepath -parameters p1parameters -analysis analysisno

(a) B's Strength

It can be seen that as the strength of B increases the probability of winning increases for both optimal policy and random policy which is consistent with intuition that with increasing strength, B won't give away his wicket thereby increasing chances of winning. Also for the optimal policy the maximum probability is not 1 which is justified as in 15 balls 30 runs are required and player A is not optimal, he can also give away his wicket and score different from what he aims for. For a fixed strength the probability of winning for optimal policy is higher then that for random policy which is explained by the fact that probability here is actually the value function at state 1530 and hence for optimal policy it will be greater than or equal to any other policy.

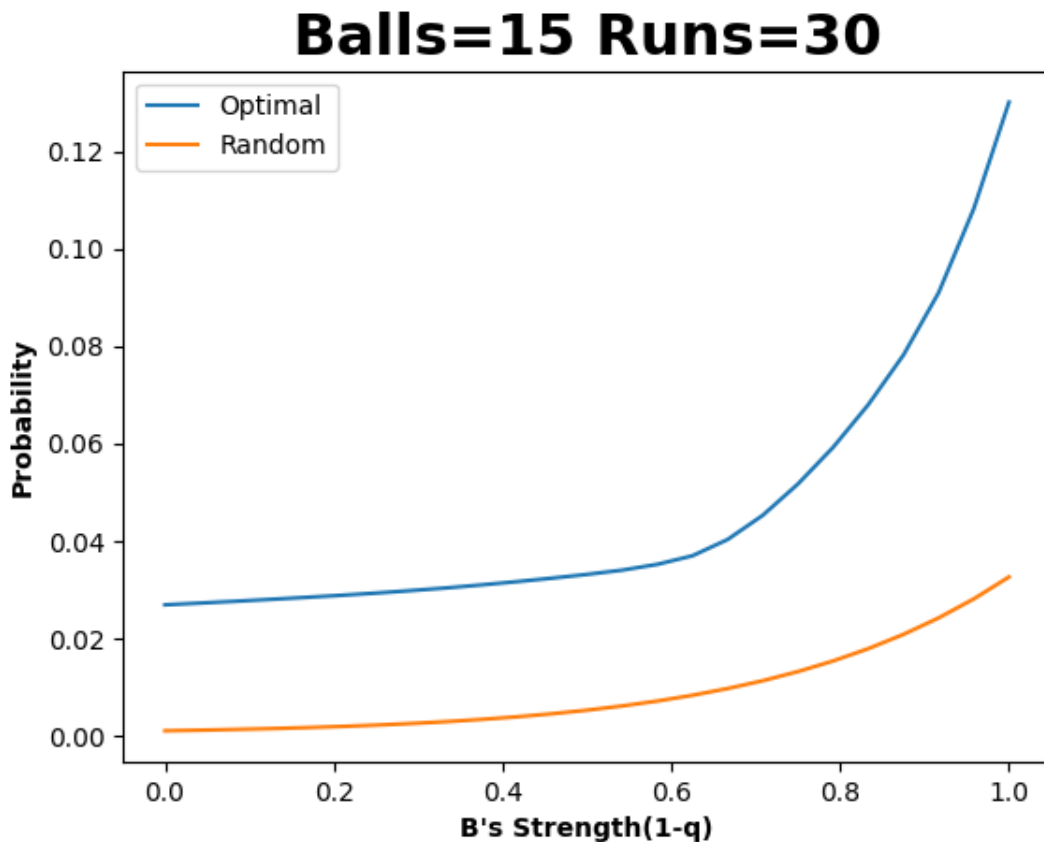


Figure 1: Win Probability vs B's Strength

(b) Fixed balls(10)

It can be seen that as the runs needed increases the probability of winning decreases for optimal policy which is consistent with our intuition that the target has got harder and for random policy it decreases on a larger scale but there are certain ups and downs which indicate that policy takes some very weird (good or bad) decisions at some states as compared to other states. Also for the optimal policy the maximum probability tends to 1 as in that case the target (10 balls 1 run) is very easy and minimum probability is not zero as the target in that case (10 balls 20 runs) is still achievable given the strength of A and B. For a fixed number of runs needed the probability of winning for optimal policy is higher than that for random policy which is explained by the fact that probability here is actually the value function at state 10rr and hence for optimal policy it will be greater than or equal to any other policy.

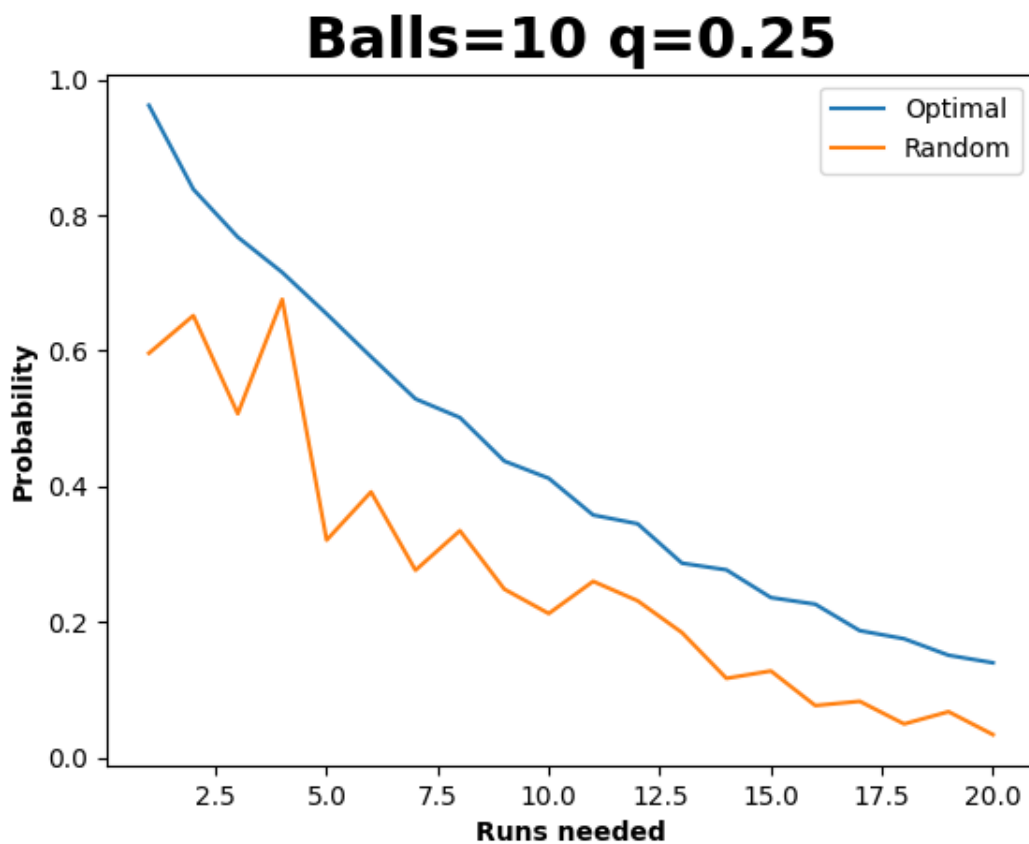


Figure 2: Win Probability vs Runs needed

(c) Fixed runs(10)

It can be seen that as the balls left increases the probability of winning increases for optimal policy on a larger scale which is consistent with our intuition that the target has got easier but for balls changing from 6 to 7 and 12 to 13 respectively the probability drops which is explained by the fact that B is quite a lot weaker ($q=0.25$) and A is not optimal and hence he might not be able to change strike even though he wishes to do so and on failing to change strike B is more likely to give away his wicket in the next over than give back strike to A. Also for random policy probability seems to increase on a larger scale but there are a lot of abrupt ups and downs which indicate that policy takes some very weird (good or bad) decisions at some states as compared to other states. Also since there are such weird decisions there is no point in trying to explain it more which can be done by taking a look at the exact policy taken. For the optimal policy the maximum probability is less than 1 as the strength of B is less and A is also not an ideal batsman and the corresponding target is not too easy. The minimum probability is 0 as the target (10 runs 1 ball) is unachievable given that we have no wides and no balls in our game. For a fixed number of balls left the probability of winning for optimal policy is higher than that for random policy (or equal) which is explained by the fact that probability here is actually the value function at state $bb10$ and hence for optimal policy it will be greater than or equal to any other policy.

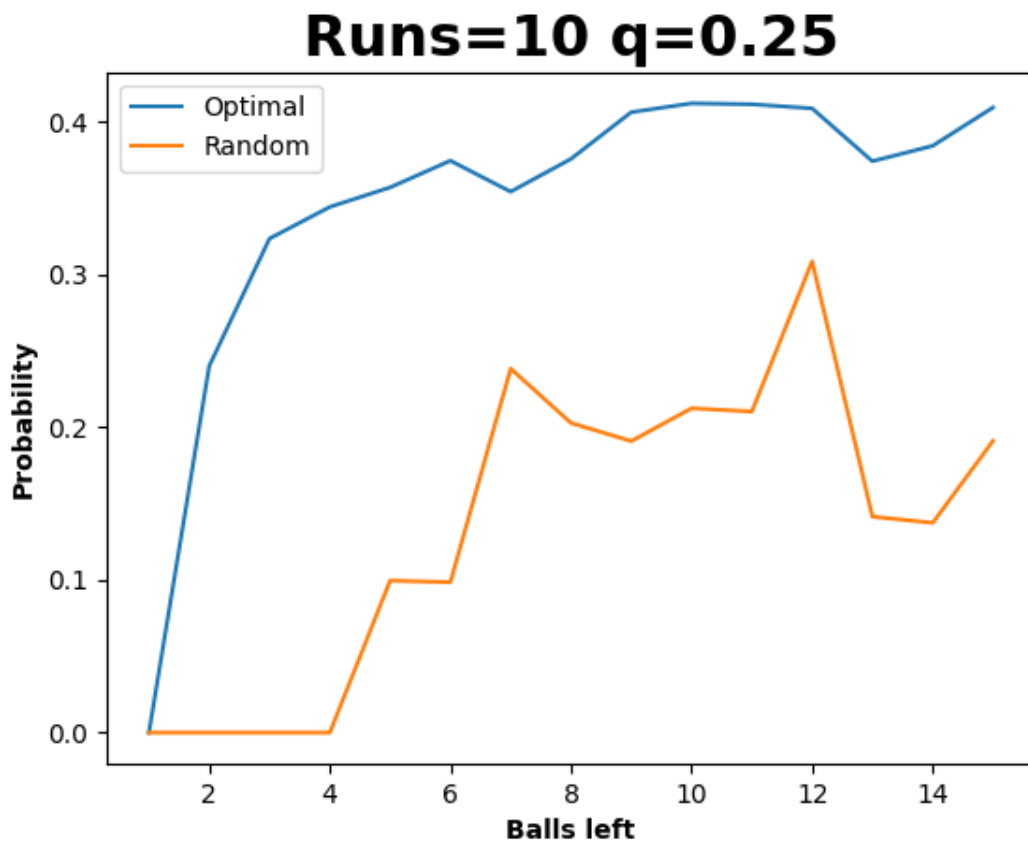


Figure 3: Win Probability vs Balls Left